KỸ THUẬT LẬP TRÌNH C#

Chương 2: Giới thiệu Visual Studio & Lập trình Trực quan

BÀI GIẢNG CHUẨN ĐẠI HỌC (3 TIẾT TÍCH HỢP)

Giáo trình: *Visual C# How to Program* (Deitel & Deitel)

Khoa Công nghệ Thông tin – Đại học Đông Á

Đối tượng: Lớp IT26M (Khóa 2026)

Học kỳ I – Năm học 2026

Bản quyền bài giảng thuộc Bộ môn Kỹ thuật Lập trình – Phục vụ đào tạo Đại học chính quy

Mục tiêu Bài học & Chuẩn đầu ra (CLOs)

Chuẩn đầu ra sinh viên cần đạt được sau buổi học 3 tiết:

- **CLO 2.1 (Understand):** Giải thích vai trò của Môi trường phát triển tích hợp (IDE) và triết lý Lập trình trực quan (Visual Programming / RAD) trong công nghiệp phần mềm.
- **CLO 2.2 (Apply):** Khởi tạo và quản trị dự án Windows Forms Application; phân tích mối quan hệ giữa các tệp `.sln`, `.csproj`, `Form.cs`, `Form.Designer.cs`.
- **CLO 2.3 (Apply):** Vận dụng thành thạo Menu Bar, Standard Toolbar, phím tắt và kỹ thuật điều hướng cửa sổ (Auto-Hide, Pinning, Docking, F1 Context-Sensitive Help).
- **CLO 2.4 (Analyze):** Khai thác tối ưu các cửa sổ nòng cốt: Solution Explorer, Toolbox (nhóm Common Controls) và Properties Window (Categorized vs Alphabetical).
- **CLO 2.5 (Create):** Vận dụng kỹ thuật kéo-thả để xây dựng hoàn chỉnh ứng dụng `ASimpleApp` (Form, Label, PictureBox với tài nguyên ảnh), tùy biến màu, font, căn lề và thực thi ở Run Mode.

Phương pháp tiếp cận:

Tích hợp thực chiến: **Lý thuyết trọng tâm** kết hợp **3 bài Hands-on Labs** thực hành trực tiếp trong phòng máy.

Tiến trình Bài học 3 Tiết (150 Phút Tích hợp)

TIẾT 1 (50')

Tổng quan IDE & Menus

- Khái niệm IDE & Triết lý RAD
- Màn hình Start Page
- Tạo New Project WinForms
- Khung nhìn Design View
- Thanh Menu Bar & Toolbars
- Trợ giúp F1 (MSDN Help)
- **[LAB 1]:** Tạo dự án & Khảo sát thanh công cụ

TIẾT 2 (50')

Cửa sổ Nòng cốt IDE

- Kỹ thuật Auto-Hide & Pinning
- Quản lý Docking & Layouts
- Solution Explorer (.sln, .csproj)
- Thư viện liên kết References
- Hộp công cụ Toolbox
- Cửa sổ Properties Window
- **[LAB 2]:** Sắp xếp layout & Chỉnh thuộc tính Form

TIẾT 3 (50')

Case Study 'ASimpleApp'

- Kiến trúc ứng dụng đồ họa
- Tùy biến Form (Text, BackColor)
- Điều khiển Label & Font Dialog
- Căn lề TextAlign & Snap Lines
- PictureBox & Project Resource
- Chế độ Design Mode vs Run Mode
- **[LAB 3]:** Hoàn thiện ứng dụng đồ họa đầu tay

Khởi động: Cuộc cách mạng từ Dòng lệnh đến Trực quan

Lập trình Dòng lệnh Cổ điển (CLI)

- Lập trình viên phải gõ tay hàng trăm dòng mã lệnh để tạo từng nút bấm, nhãn chữ, tọa độ pixels.
- Không thể nhìn thấy trước giao diện thực tế cho đến khi biên dịch và chạy thử.
- Dễ sai sót về bố cục, tốn rất nhiều thời gian định vị tọa độ (`x, y, w, h`).

Lập trình Trực quan (Visual Programming)

- Áp dụng nguyên lý **WYSIWYG** (*What You See Is What You Get*).
- Thiết kế giao diện bằng cách kéo-thả (Drag-and-Drop) điều khiển từ Toolbox vào Form.
- IDE tự động sinh mã nguồn khởi tạo (`Form.Designer.cs`) ngầm bên dưới.
- Tăng tốc độ phát triển ứng dụng lên gấp 5–10 lần (**Rapid Application Development - RAD**).

Tâm thế kỹ sư IT26M:

Chúng ta không chỉ học cách ”vẽ giao diện”, mà học cách làm chủ kiến trúc sinh mã tự động của Visual Studio và tư duy hướng đối tượng phía sau các điều khiển đồ họa!

TIẾT 1

TỔNG QUAN VISUAL STUDIO IDE, CẤU TRÚC DỰ ÁN & MENU / TOOLBAR

Môi trường Phát triển Tích hợp (IDE) là gì?

Định nghĩa IDE (Integrated Development Environment):

Là gói phần mềm chuyên dụng cung cấp toàn diện các công cụ cần thiết cho toàn bộ vòng đời phát triển phần mềm trên một giao diện thống nhất.

Các thành phần cốt lõi của Visual Studio IDE:

1. **Text Editor thông minh:** Tự động đổi màu cú pháp, thụt lề, gợi ý mã lệnh (IntelliSense).
2. **Visual Designer:** Không gian thiết kế giao diện đồ họa kéo-thả trực quan.
3. **Trình biên dịch tích hợp (Compiler):** Tự động biên dịch mã C# sang MSIL và gọi JIT.
4. **Trình gỡ lỗi nâng cao (Debugger):** Đặt điểm dừng (Breakpoints), theo dõi biến số (Watch).

Các phiên bản Visual Studio:

- **Community:** Bản miễn phí đầy đủ tính năng dành cho sinh viên, giảng viên và dự án cá nhân.
- **Professional:** Bản thương mại cho doanh nghiệp vừa và nhỏ.
- **Enterprise:** Bản cao cấp nhất cho tập đoàn lớn, tích hợp kiểm thử tải và kiến trúc nâng cao.

Triết lý Phát triển Ứng dụng Nhanh (RAD)

Đặc trưng của phương pháp RAD trong C# WinForms:

- **Mô hình hóa nhanh (Prototyping):** Xây dựng bản mẫu giao diện hoàn chỉnh chỉ trong vài phút để lấy ý kiến khách hàng.
- **Tái sử dụng linh kiện phần mềm (Component Reuse):** Tận dụng hàng trăm điều khiển có sẵn trong thư viện .NET Framework Class Library (FCL).
- **Tách biệt giao diện và mã xử lý:**
  - Tệp `Form.Designer.cs`: Chứa mã giao diện tự sinh.
  - Tệp `Form.cs`: Chứa mã xử lý logic do lập trình viên viết.

Lợi thế cạnh tranh của RAD:

Tiết kiệm thời gian

⇓

Giảm chi phí phát triển

⇓

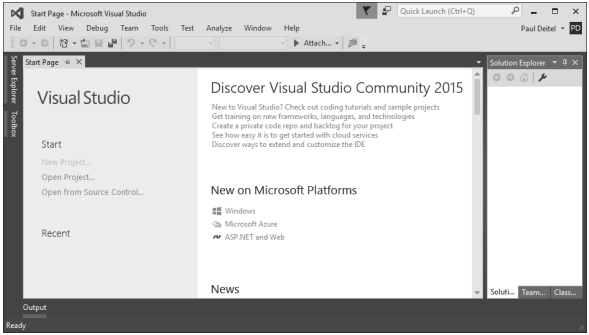
Tập trung vào logic cốt lõi

Thay vì viết mã lặp đi lặp lại để vẽ nút bấm và khung cửa sổ!

Khởi động Visual Studio & Màn hình Start Page

Các khu vực chính trên Start Page:

- **Start Section:** Các thao tác mở đầu:
  - **New Project...** (Ctrl+Shift+N): Tạo giải pháp mới.
  - **Open Project...** (Ctrl+Shift+O): Mở dự án có sẵn.
- **Recent Projects:** Danh sách các giải pháp vừa làm việc gần đây, cho phép ghim cố định (Pin to list).
- **Discover / News Feed:** Cập nhật tài liệu, video hướng dẫn và tin tức kỹ thuật từ Microsoft Developer Network.



Hình 2.1: Màn hình Start Page trong Visual Studio (Giáo trình Deitel, trang 3).

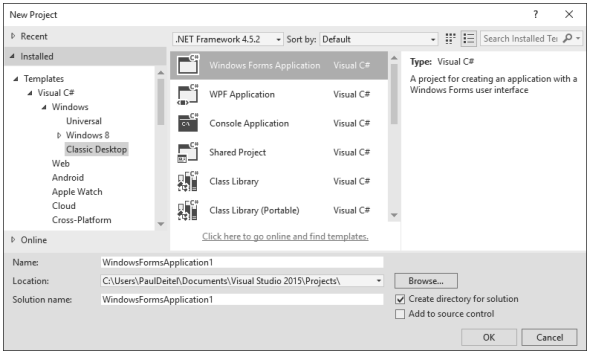
Hộp thoại Tạo Dự án Mới (New Project Dialog)

Quy trình 4 bước tạo dự án chuẩn:

1. **Chọn Ngôn ngữ:** Chọn **Visual C#** ở ngăn phân cấp bên trái.
2. **Chọn Mẫu ứng dụng:** Chọn **Windows Forms Application** ở ngăn giữa.
3. **Đặt Tên dự án (Name):** Đặt tên gợi nhớ theo chuẩn PascalCase (ví dụ: `ASimpleApp`).
4. **Chọn Thư mục lưu trữ (Location):** Trỏ đến thư mục bài tập của sinh viên (ví dụ: `D:\CSharpLabs\`).

Lưu ý quan trọng:

Đánh dấu chọn *"Create directory for solution"* để tạo thư mục Solution độc lập.

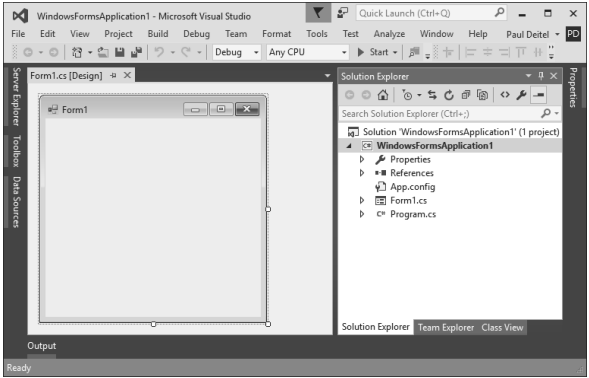


Hình 2.3: Hộp thoại New Project dialog trong Visual Studio (Giáo trình Deitel, trang 5).

Khung nhìn Thiết kế Trực quan (Design View)

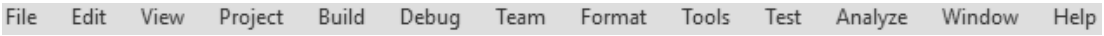
Cấu trúc không gian làm việc IDE:

- **Form Designer (ở giữa):** Vùng làm việc chính hiển thị biểu mẫu Form rỗng màu xám với tiêu đề mặc định `Form1`.
- **Toolbox (bên trái):** Chứa các điều khiển đồ họa sẵn sàng để kéo thả.
- **Solution Explorer (góc trên bên phải):** Quản lý cây tệp tin và cấu trúc dự án.
- **Properties Window (góc dưới bên phải):** Bảng thuộc tính của đối tượng đang chọn.



Hình 2.4: Khung nhìn Design View khi mở dự án mới (Giáo trình Deitel, trang 6).

Thanh Menu Bar – Trung tâm Điều khiển Lệnh của IDE



Hình 2.5: Thanh Menu Bar trong Visual Studio (Giáo trình Deitel, trang 7).

Menu	Chức năng cốt lõi	Menu	Chức năng cốt lõi
File	Tạo, mở, lưu, đóng tệp và dự án	**Debug**	Khởi chạy (F5), gỡ lỗi, Step Over (F10)
Edit	Cut, Copy, Paste, Undo, Find & Replace	**Format**	Căn hàng, dàn đều khoảng cách điều khiển
View	Bật/tắt các cửa sổ (Toolbox, Properties)	**Tools**	Tùy biến cài đặt IDE, kết nối CSDL
Project	Thêm Form mới, Class mới, Add Reference	**Window**	Bố trí cửa sổ (Split, Tabbed, Float)
Build	Biên dịch dự án (Build Solution / Rebuild)	**Help**	Tra cứu tài liệu, mẫu mã, phím F1

Chi tiết các Menu Quan trọng: Build, Debug & Format

Menu BUILD

- **Build Solution (Ctrl+Shift+B):** Biên dịch toàn bộ các tệp mã nguồn trong giải pháp ra tệp `.exe` hoặc `.dll`.
- **Rebuild Solution:** Xóa sạch kết quả cũ và biên dịch lại từ đầu mọi tệp tin.
- **Clean Solution:** Xóa các tệp trung gian trong thư mục `bin/` và `obj/`.

Menu DEBUG

- **Start Debugging (F5):** Chạy ứng dụng có gắn trình gỡ lỗi (bắt lỗi Runtime).
- **Start Without Debugging (Ctrl+F5):** Chạy ứng dụng độc lập với tốc độ tối đa.
- **Toggle Breakpoint (F9):** Đặt điểm dừng trên dòng lệnh để kiểm tra biến.

Menu FORMAT

- Chỉ xuất hiện khi Form đang ở **Design View**.
- **Align:** Căn bằng mép trái, phải, trên, dưới giữa các điều khiển.
- **Make Same Size:** Đồng bộ kích thước Width/Height tự động.
- **Center in Form:** Căn chính giữa Form theo chiều ngang/dọc.

Thanh Công cụ Chuẩn (Standard Toolbar)

Các nút bấm thao tác nhanh:

- **New Project / Open File:** Phím tắt mở nhanh.
- **Save (Ctrl+S) / Save All (Ctrl+Shift+S):** Lưu tệp hiện tại hoặc lưu toàn bộ giải pháp.
- **Undo (Ctrl+Z) / Redo (Ctrl+Y):** Hoàn tác hoặc làm lại thao tác thiết kế.
- **Start Button (F5):** Nút mũi tên màu xanh lá để kích hoạt biên dịch và chạy thử.
- **Solution Configurations:** Chuyển đổi giữa chế độ `Debug` (gỡ lỗi) và `Release` (đóng gói phát hành).



Hình 2.7: Thanh Standard Toolbar trong Visual Studio (Deitel, trang 8).

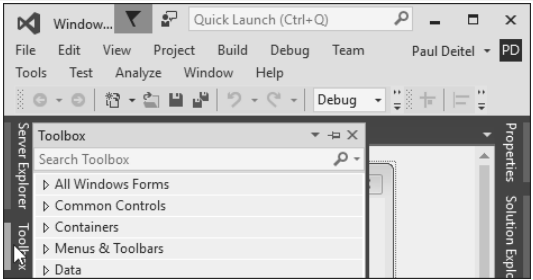
Mẹo tăng tốc:

Luôn duy trì thói quen nhấn **Ctrl+Shift+S** (Save All) trước khi chạy thử ứng dụng!

Tính năng Tooltips & Bảng Phím tắt Thường dùng

Tooltips (Chú giải công cụ)

- Khi rê chuột qua bất kỳ nút bấm nào trên thanh công cụ, một hộp thoại văn bản nhỏ sẽ nổi lên.
- Cung cấp: **Tên chức năng** kèm theo **Tổ hợp phím tắt tương ứng**.



Hình 2.11: Hiển thị cửa sổ ẩn khi rê chuột.

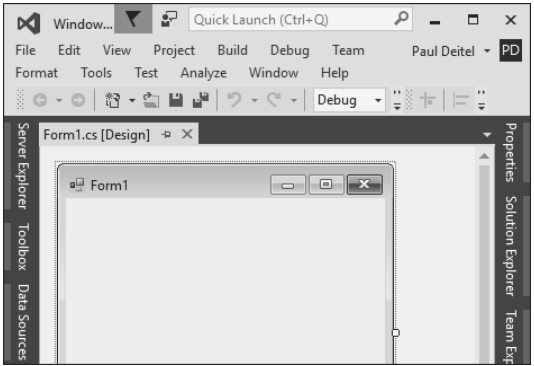
Bảng Phím tắt Kỹ sư IT26M cần thuộc:

Phím tắt	Hành động tương ứng
F5	Start Debugging (Chạy có gỡ lỗi)
Ctrl + F5	Start Without Debugging (Chạy nhanh)
Shift + F5	Dừng chương trình đang chạy
F4	Bật cửa sổ Properties Window
Ctrl + Alt + X	Bật cửa sổ Toolbox
Ctrl + Alt + L	Bật cửa sổ Solution Explorer
F7 / Shift + F7	Chuyển đổi giữa Code View & Designer
Ctrl + Shift + B	Biên dịch dự án (Build Solution)

Hệ thống Trợ giúp Ngữ cảnh (Context-Sensitive Help) & Phím F1

Sức mạnh của phím F1:

- Khi đang chọn bất kỳ điều khiển GUI nào (ví dụ: Button, PictureBox) hoặc một từ khóa C# trong Code Editor:
- **Nhấn phím F1:** Visual Studio sẽ tự động kích hoạt trình duyệt và điều hướng trực tiếp tới trang tài liệu chính thức của thành phần đó trên **Microsoft Learn / MSDN Library**.
- Sinh viên không cần tốn công gõ tìm kiếm thủ công trên Google.



Hình 2.10: Cơ chế tương tác trực quan với các thành phần IDE.

Văn hóa đọc tài liệu:

Kỹ sư phần mềm xuất sắc là người biết cách khai thác tài liệu chuẩn Microsoft để tự học thuộc tính và sự kiện mới!

Checkpoint 1: Kiểm tra Kiến thức Nhanh Tiết 1

Câu hỏi 1:

Tổ hợp phím tắt nào trong Visual Studio cho phép lập trình viên chạy ứng dụng ở chế độ độc lập với tốc độ cao nhất (không gắn trình gỡ lỗi Debugger)?

(A) Phím **F5**

(B) Tổ hợp phím **Ctrl + F5**

(C) Tổ hợp phím **Ctrl + Shift + B**

(D) Phím **F4**

Checkpoint 1: Kiểm tra Kiến thức Nhanh Tiết 1

Câu hỏi 1:

Tổ hợp phím tắt nào trong Visual Studio cho phép lập trình viên chạy ứng dụng ở chế độ độc lập với tốc độ cao nhất (không gắn trình gỡ lỗi Debugger)?

(A) Phím **F5**

(B) Tổ hợp phím **Ctrl + F5**

(C) Tổ hợp phím **Ctrl + Shift + B**

(D) Phím **F4**

Giải thích đáp án:

Đáp án chính xác: B (Ctrl + F5). Phím F5 kích hoạt *Start Debugging* (chạy có gỡ lỗi, kiểm tra bộ nhớ), trong khi **Ctrl + F5** là *Start Without Debugging* giúp ứng dụng khởi chạy độc lập tức thì.

Hands-on Lab 1: Khởi tạo Solution & Khảo sát Menus (15 Phút)

NHIỆM VỤ THỰC HÀNH TẠI PHÒNG MÁY:

1. Khởi động Visual Studio Community trên máy tính phòng máy.
2. Chọn **File** → **New** → **Project...** (hoặc Ctrl+Shift+N).
3. Chọn ngôn ngữ **Visual C#** → **Windows Forms Application**.
4. Đặt tên dự án: `Lab01_VisualIntro_IT26M`.
5. Lưu vào thư mục cá nhân: `D:\IT26M_Labs\`.
6. Khảo sát thanh Menu Bar và Standard Toolbar.
7. Nhấn **Ctrl + F5** để chạy thử biểu mẫu Form rỗng đầu tiên.

KẾT QUẢ MONG ĐỢI:

- Cửa sổ Form1 màu xám xuất hiện ở giữa màn hình.
- Có đầy đủ 3 nút tiêu chuẩn trên thanh tiêu đề: Minimize, Maximize và Close (X).
- Kéo giãn cửa sổ bằng chuột để xác nhận Form hoạt động bình thường.
- Đóng cửa sổ để quay lại không gian thiết kế của Visual Studio.

TIẾT 2

ĐIỀU HƯỚNG CỬA SỔ IDE, SOLUTION EXPLORER, TOOLBOX & PROPERTIES

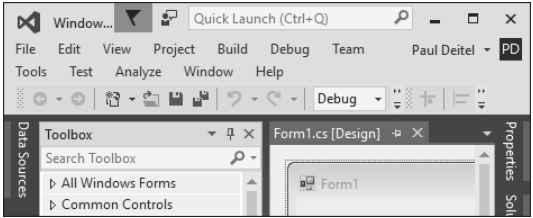
Kỹ thuật Điều hướng: Tính năng Auto-Hide & Nút Ghim (Pin Icon)

Cơ chế Tự động Ẩn (Auto-Hide):

- Khi không gian màn hình chật hẹp, ta có thể thu nhỏ các cửa sổ công cụ thành các tab chữ dọc mép màn hình.
- Khi rê chuột lên tab (ví dụ: `Toolbox`), cửa sổ sẽ trượt ra nhẹ nhàng để ta chọn điều khiển.
- Khi đưa chuột ra ngoài, cửa sổ tự động ẩn lại để giải phóng không gian cho Form Designer.

Nút Ghim (Pin Icon):

- **Cây ghim nằm ngang:** Cửa sổ đang ở chế độ *Auto-Hide*.
- **Cây ghim thẳng đứng:** Cửa sổ được *"ghim chặt"* cố định trên màn hình làm việc.



Hình 2.12: Nút cây ghim (Pin icon) để tắt tính năng Auto-Hide (Deitel, trang 11).

Kỹ thuật Ghép nối Cửa sổ (Docking & Tab Groups)

Kỹ thuật Docking chuyên nghiệp:

- Kéo thanh tiêu đề của bất kỳ cửa sổ nào, Visual Studio sẽ hiển thị **Bánh dẫn đường Docking Guide** hình chữ thập.
- Kéo chuột vào mũi tên:
  - **Top / Bottom / Left / Right:** Chia đôi không gian màn hình theo hướng tương ứng.
  - **Center:** Nhập cửa sổ thành một tab chung với cửa sổ hiện có.

Khôi phục bố cục mặc định:

Nếu lỡ tay làm lộn xộn cửa sổ, chỉ cần vào: **Window** → **Reset Window Layout** để trở về trạng thái chuẩn ban đầu!

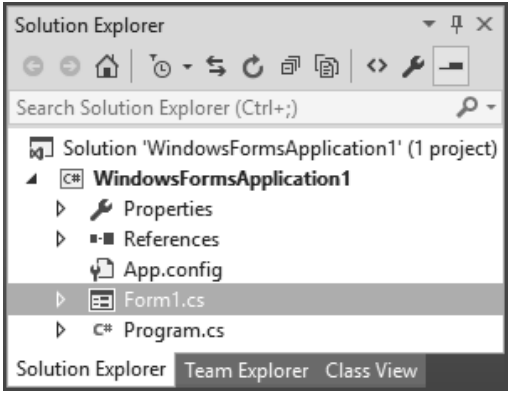
3 Trạng thái Hiển thị Cửa sổ:

1. **Docked (Neo cố định):** Gắn chặt vào một mép của khung nhìn IDE.
2. **Floating (Trôi nổi tự do):** Kéo rời ra ngoài thành một cửa sổ Windows độc lập (tiện lợi khi dùng 2 màn hình).
3. **Tabbed Document (Thẻ văn bản):** Xếp chồng thành các tab tài liệu (Form1.cs, Form1.Designer.cs).

Cửa sổ Solution Explorer: Cấu trúc Giải pháp & Dự án

Phân cấp Solution & Project:

- **Solution (Tệp `.sln`):** Container cấp cao nhất, có thể quản lý một hoặc nhiều dự án liên quan cùng các thiết lập biên dịch chung.
- **Project (Tệp `.csproj`):** Đơn vị cấu thành phần mềm, chứa các tệp mã nguồn C#, tài nguyên hình ảnh, biểu tượng và cấu hình dependencies.
- Cho phép kích chuột phải để thêm tệp mới, đổi tên hoặc thiết lập dự án khởi động (*Set as Startup Project*).



Hình 2.13: Cửa sổ Solution Explorer (Giáo trình Deitel, trang 11).

Giải phẫu các Tệp tin trong Dự án Windows Forms

Tên Tệp Tin / Nút	Ý nghĩa Kỹ thuật & Vai trò
Solution (.sln)	Tệp văn bản quản lý danh sách các Project và cấu hình build (Debug/Release).
Project (.csproj)	Tệp XML đặc tả các tệp mã nguồn, phiên bản .NET SDK và các gói thư viện NuGet.
Properties / App.config	Chứa thông tin bản quyền ứng dụng, chuỗi kết nối cơ sở dữ liệu và cài đặt cấu hình.
References	Danh sách các thư viện liên kết động (.dll) của .NET Framework mà dự án sử dụng.
Form1.cs	Tệp mã nguồn C# chứa logic ứng dụng và các hàm xử lý sự kiện (Event Handlers) do lập trình viên viết.
Form1.Designer.cs	Tệp C# do Visual Studio tự động sinh ra và cập nhật mỗi khi lập trình viên kéo-thả điều khiển trên Form.
Form1.resx	Tệp tài nguyên XML lưu trữ hình ảnh, chuỗi đa ngôn ngữ nhúng bên trong Form.
Program.cs	Tệp chứa điểm vào (Entry Point) của ứng dụng – hàm `static void Main()` khởi chạy Form.

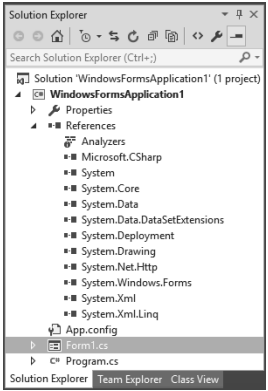
Cảnh báo vàng:

Sinh viên tuyệt đối **không sửa tay trực tiếp** nội dung bên trong tệp `Form1.Designer.cs` trừ khi thực sự hiểu rõ, vì IDE có thể ghi đè làm hỏng giao diện Form Designer!

Nút References – Quản lý Thư viện Liên kết (Assemblies)

Vai trò của nút References:

- Một ứng dụng C# không thể chạy độc lập nếu thiếu các thư viện lớp cơ sở của .NET (Framework Class Library).
- Nút **References** hiển thị tất cả các Assembly (.dll) được nạp vào dự án:
  - `System`: Các kiểu dữ liệu cơ bản, ngoại lệ, I/O.
  - `System.Drawing`: Xử lý đồ họa, màu sắc, font chữ.
  - `System.Windows.Forms`: Chứa các lớp điều khiển GUI (Form, Button, TextBox...).



Hình 2.14: Cây References được mở rộng (Giáo trình Deitel, trang 12).

Cửa sổ Toolbox – Kho Linh kiện Điều khiển Đồ họa

Cấu trúc phân loại trong Toolbox:

- Chứa tất cả các điều khiển giao diện (Controls) có thể đưa lên Form.
- Phân chia thành các nhóm chủ đề có thể thu gọn/mở rộng:
  - **Common Controls:** Các điều khiển phổ biến nhất (Button, Label, TextBox, CheckBox, RadioButton, PictureBox...).
  - **Containers:** Khung chứa phân nhóm điều khiển (GroupBox, Panel, TabControl...).
  - **Menus & Toolbars:** Tạo hệ thống menu thả và thanh công cụ.
  - **Data:** Điều khiển hiển thị dữ liệu lưới (DataGridView).

2 Cách đưa điều khiển lên Form:

1. **Cách 1 (Kéo-Thả):** Nhấn giữ chuột trái vào điều khiển trong Toolbox, kéo rê vào Form rồi thả ra tại vị trí mong muốn.
2. **Cách 2 (Nhấp đúp chuột):** Nhấp đúp chuột trái vào biểu tượng điều khiển trong Toolbox, điều khiển sẽ tự động xuất hiện ở góc trên bên trái của Form.

Khám phá Nhóm Common Controls Thông dụng



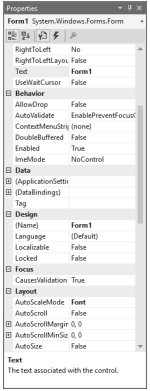
Hình 2.15: Nhóm Common Controls trong Toolbox (Deitel, trang 13).

Điều khiển	Công dụng trực quan chính
Label	Hiển thị nhãn văn bản cố định, hướng dẫn người dùng.
TextBox	Hộp cho phép người dùng nhập dữ liệu văn bản từ bàn phím.
Button	Nút bấm kích hoạt hành động khi người dùng nhấp chuột.
PictureBox	Khung chuyên dụng hiển thị hình ảnh đồ họa (BMP, JPG, PNG).
CheckBox	Nút chọn nhiều mục độc lập (Bật / Tắt trạng thái).
RadioButton	Nút chọn tròn độc quyền (chỉ chọn 1 trong nhóm).

Cửa sổ Properties Window: Cấu hình Lúc Thiết kế (Design-Time)

Cấu trúc của Properties Window:

- **Object Dropdown List (ở trên cùng):** Hiển thị tên định danh và kiểu của điều khiển đang được chọn (ví dụ: `Form1 System.Windows.Forms.Form`).
- **Thanh công cụ chuyển chế độ xem:**
  - Categorized (phân nhóm) vs Alphabetical (A-Z).
  - Properties (thuộc tính) vs Events (sự kiện - biểu tượng tia sét).
- **Bảng 2 cột:** Cột trái là **Tên thuộc tính**, cột phải là **Giá trị thiết lập**.
- **Description Pane (ở đáy):** Tóm tắt ngắn gọn ý nghĩa của thuộc tính đang chọn.



Hình 2.16: Cửa sổ Properties Window của Form1 (Deitel, trang 14).

Hai Chế độ Xem Properties: Alphabetical vs Categorized

Chế độ Categorized (Phân nhóm theo chủ đề)

- Gom nhóm các thuộc tính có cùng tính chất kỹ thuật:
  - **Appearance:** BackColor, Font, ForeColor, Text.
  - **Behavior:** Enabled, Visible.
  - **Design:** (Name) – Tên biến định danh điều khiển.
  - **Layout:** Location, Size, AutoSize, Dock, Anchor.
- Thích hợp cho người mới bắt đầu làm quen để hiểu phân loại.

Chế độ Alphabetical (Sắp xếp A → Z)

- Xếp toàn bộ danh sách thuộc tính theo thứ tự bảng chữ cái từ A đến Z (ngoại trừ thuộc tính `(Name)` luôn đặt ở đầu).
- Được các kỹ sư phần mềm chuyên nghiệp ưa chuộng vì tìm kiếm cực nhanh khi đã biết rõ tên thuộc tính cần chỉnh (ví dụ: gõ chữ ‘T‘ để nhảy ngay đến ‘Text‘).

Checkpoint 2: Kiểm tra Kiến thức Nhanh Tiết 2

Câu hỏi 2:

Tệp tin nào sau đây chứa toàn bộ mã nguồn khởi tạo điều khiển đồ họa do Visual Studio tự động sinh ra khi lập trình viên thực hiện thao tác kéo-thả trên Form Designer?

(A) Tệp `Program.cs`

(B) Tệp `Form1.cs`

(C) Tệp `Form1.Designer.cs`

(D) Tệp cấu hình `App.config`

Checkpoint 2: Kiểm tra Kiến thức Nhanh Tiết 2

Câu hỏi 2:

Tệp tin nào sau đây chứa toàn bộ mã nguồn khởi tạo điều khiển đồ họa do Visual Studio tự động sinh ra khi lập trình viên thực hiện thao tác kéo-thả trên Form Designer?

(A) Tệp `Program.cs`

(B) Tệp `Form1.cs`

(C) Tệp `Form1.Designer.cs`

(D) Tệp cấu hình `App.config`

Giải thích đáp án:

Đáp án chính xác: C (`Form1.Designer.cs`). Visual Studio áp dụng cơ chế lớp bộ phận (`partial class`) để tách biệt hoàn toàn mã giao diện tự sinh (`Form1.Designer.cs`) ra khỏi mã logic xử lý sự kiện của lập trình viên (`Form1.cs`).

Hands-on Lab 2: Sắp xếp Giao diện & Tùy biến Form (15 Phút)

NHIỆM VỤ THỰC HÀNH TẠI CHỖ:

1. Trên dự án `Lab01`, mở cửa sổ **Properties Window** (phím F4).
2. Chuyển đổi qua lại giữa hai chế độ xem **Alphabetical** và **Categorized**.
3. Tìm thuộc tính **Text** của Form, đổi giá trị từ `Form1` thành:

   "Học phần Kỹ thuật Lập trình IT26M"
4. Tìm thuộc tính **StartPosition**, chọn giá trị: `CenterScreen`.
5. Bật tính năng **Auto-Hide** trên cửa sổ Toolbox, sau đó dùng nút ghim (Pin icon) để cố định lại.

KẾT QUẢ KIỂM CHỨNG:

- Thanh tiêu đề của Form trên màn hình Design View lập tức đổi thành chuỗi tiếng Việt mới.
- Nhấn **Ctrl + F5**: Cửa sổ chương trình xuất hiện ngay chính giữa màn hình máy tính nhờ thuộc tính `CenterScreen`.
- Sinh viên làm chủ kỹ thuật ghim/thả các cửa sổ làm việc không bị lúng túng khi thao tác.

TIẾT 3

CASE STUDY THỰC HÀNH ‘ASimpleApp‘ (TEXT & IMAGE) & KIỂM THỬ

Giới thiệu Case Study: Xây dựng Ứng dụng ‘ASimpleApp‘

Mục tiêu Case Study (Mục 2.6 SGK):

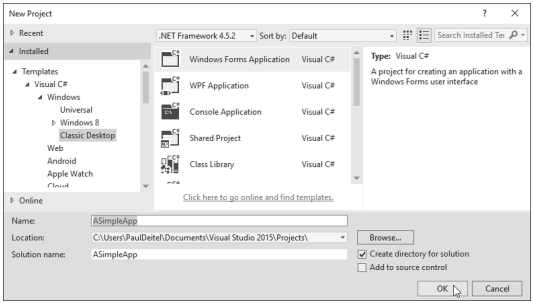
- Xây dựng một ứng dụng đồ họa Windows Forms hoàn chỉnh từ đầu mà **chưa cần viết một dòng mã C# nào**.
- Rèn luyện kỹ năng sử dụng thành thạo:
  1. Điều khiển **Form** (biểu mẫu nền, màu sắc, tiêu đề).
  2. Điều khiển **Label** (hiển thị văn bản chào mừng, tùy biến Font).
  3. Điều khiển **PictureBox** (hiển thị hình ảnh linh vật có co giãn).



Hình 2.17: Mục tiêu thành phẩm: Ứng dụng ASimpleApp khi chạy (Deitel, trang 15).

Bước 1: Khởi tạo Dự án ‘ASimpleApp‘ trong Visual Studio

1. Mở Visual Studio, chọn **File** → **New** → **Project...**
2. Chọn mẫu **Visual C#** → **Windows Forms Application**.
3. Tại ô **Name**: Nhập chính xác tên dự án là `ASimpleApp`.
4. Tại ô **Location**: Trỏ vào thư mục bài tập của sinh viên.
5. Nhấn nút **OK** để IDE tự động khởi tạo khung giải pháp mới.



Hình 2.18: Đặt tên dự án ASimpleApp trong New Project dialog (Deitel, trang 16).

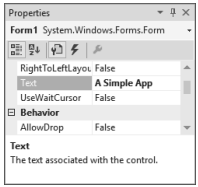
Bước 2: Tùy biến Tiêu đề Form & Kéo giãn Kích thước

Đổi Tiêu đề Form (Text Property):

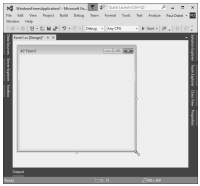
- Chọn Form trên màn hình Design View.
- Trong cửa sổ Properties, tìm thuộc tính **Text**.
- Nhập giá trị mới: **`A Simple App`**.
- Tiêu đề trên thanh tiêu đề Form lập tức cập nhật.

Điểm mút thay đổi kích thước (Sizing Handles):

- Quanh viền Form xuất hiện 8 chấm vuông nhỏ màu trắng gọi là *sizing handles*.
- Rê chuột vào chấm góc dưới bên phải, con trỏ chuột chuyển thành mũi tên chéo hai đầu (↘).
- Kéo chuột để mở rộng kích thước Form.



Hình 2.19: Đổi thuộc tính Text của Form (Deitel, trang 16).

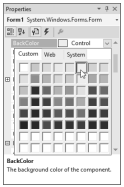


Hình 2.21: Thao tác kéo giãn Form qua sizing handle (Deitel, trang 17).

Bước 3: Tùy biến Màu nền Form (BackColor Property)

Cấu hình thuộc tính BackColor:

- Tìm thuộc tính **BackColor** trong Properties Window.
- Nhấp vào mũi tên thả xuống ở cột giá trị để mở **Color Palette**:
  - **Custom:** Chọn màu tùy ý theo mã RGB.
  - **Web:** Bảng tên màu web chuẩn quốc tế.
  - **System:** Màu theo chủ đề hệ điều hành Windows.
- Chọn tab **Web** → chọn màu **LightCyan** (hoặc màu xanh nhạt).



Hình 2.22: Bảng chọn màu BackColor (Deitel, trang 18).



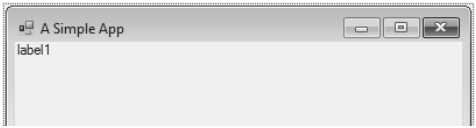
Hình 2.23: Form sau khi nhận màu nền LightCyan (Deitel, trang 18).

Bước 4: Thêm Điều khiển Label & Đổi Nội dung Văn bản

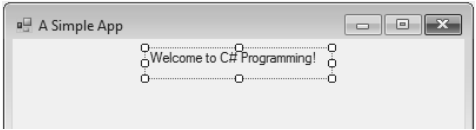
Thao tác thêm Label:

- Mở cửa sổ **Toolbox** → nhóm **Common Controls**.
- Kéo thả điều khiển **Label** vào vị trí phía trên cùng của Form.
- Điều khiển xuất hiện với tên mặc định là `label1`.
- Trong Properties Window, tìm thuộc tính **Text**.
- Nhập nội dung mới:

```
Welcome to Visual C#!
```



Hình 2.24: Kéo thả Label từ Toolbox vào Form (Deitel, trang 19).

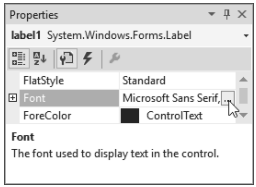


Hình 2.25: Giao diện sau khi đổi chuỗi chào mừng (Deitel, trang 19).

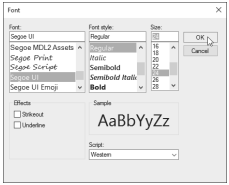
Bước 5: Tùy biến Font Chữ qua Hộp thoại Font Dialog

Cấu hình thuộc tính Font của Label:

- Chọn điều khiển Label trên Form.
- Trong Properties Window, tìm thuộc tính **Font**.
- Nhấp vào nút có dấu ba chấm **[...]** ở bên phải giá trị để mở **Font Dialog**.
- Thiết lập các thông số:
  - **Font:** `Segoe UI` (hoặc Tahoma).
  - **Font style: Bold** (Đậm).
  - **Size: 24** points.
- Nhấn **OK** để áp dụng font chữ lớn, sắc nét.



Hình 2.26: Nút mở rộng thuộc tính Font (Deitel, trang 20).

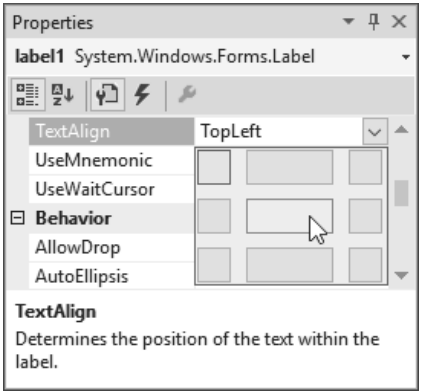


Hình 2.27: Hộp thoại Font dialog chuẩn (Deitel, trang 20).

Bước 6: Căn giữa Văn bản với Thuộc tính TextAlign

Thiết lập Căn lề Hoàn hảo:

- Đặt thuộc tính **AutoSize** của Label thành **False** để cho phép tự do kéo giãn kích thước khung Label theo chiều ngang bằng Form.
- Tìm thuộc tính **TextAlign** trong Properties Window.
- Nhấp chuột vào biểu tượng lưới 9 ô vuông:
  - Chọn ô **chính giữa** tương ứng giá trị `MiddleCenter`.
- Dòng chữ *"Welcome to Visual C#!"* lập tức nằm cân đối chính giữa khung Form!



Hình 2.28: Lựa chọn MiddleCenter trên lưới thuộc tính TextAlign (Deitel, trang 21).

Bước 7: Thêm Điều khiển PictureBox & Đường Căn Snap Lines

Đường căn thông minh (Snap Lines):

- Kéo thả điều khiển **PictureBox** từ Toolbox đặt vào vùng trống bên dưới Label.
- Trong lúc kéo rê, Visual Studio tự động hiển thị các đường kẻ màu xanh dương (Snap lines) chỉ báo khoảng cách tiêu chuẩn giữa điều khiển với viền Form và các điều khiển lân cận.
- Giúp giao diện ứng dụng luôn gọn gàng, chuẩn thẩm mỹ mà không cần đo đạc thủ công.

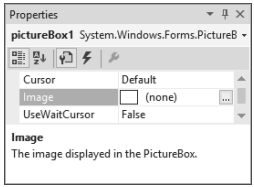


Hình 2.29: Căn chỉnh PictureBox bằng đường Snap Lines (Deitel, trang 21).

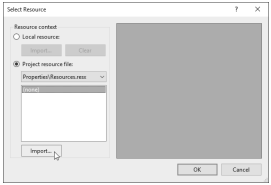
Bước 8: Nạp Hình ảnh Tài nguyên (Select Resource Dialog)

Quy trình nhúng ảnh vào dự án:

- Chọn PictureBox → tìm thuộc tính **Image** trong Properties Window.
- Nhấp nút **[...]** để mở hộp thoại **Select Resource**.
- Chọn tùy chọn **Project resource file** để đóng gói ảnh vĩnh viễn vào tệp thực thi `.exe`.
- Nhấn nút **Import...** → duyệt chọn tệp hình ảnh linh vật `bug.png` từ thư mục tài nguyên.
- Nhấn **OK** để hoàn tất gán ảnh.



Hình 2.30: Thuộc tính Image của PictureBox (Deitel, trang 22).



Hình 2.31: Hộp thoại Select Resource (Deitel, trang 22).

Bước 9: Co giãn Hình ảnh với Thuộc tính SizeMode

Vấn đề thường gặp:

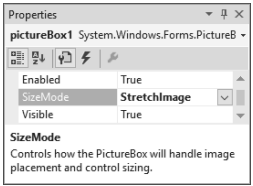
Kích thước ảnh gốc thường lớn hơn khung PictureBox, khiến ảnh bị cắt xén (chỉ thấy một góc nhỏ).

Các giá trị của thuộc tính SizeMode:

- **Normal (mặc định):** Đặt ảnh ở góc trên bên trái, phần thừa bị cắt.
- **StretchImage:** Co/giãn ảnh vừa khít toàn bộ chiều rộng và chiều cao của khung PictureBox.
- **AutoSize:** Khung PictureBox tự phình to bằng đúng kích thước ảnh gốc.
- **CenterImage:** Căn ảnh vào chính giữa khung PictureBox.
- **Zoom:** Co giãn ảnh vừa khung nhưng giữ nguyên tỷ lệ khung hình (*Aspect Ratio*) không bị méo.



Hình 2.33: Chọn chế độ StretchImage (Deitel, trang 23).



Hình 2.34: Hình ảnh hiển thị hoàn hảo vừa vặn khung (Deitel, trang 23).

Bước 10: Chạy Ứng dụng & Phân biệt Design vs Run Mode

Khởi chạy ứng dụng:

- Nhấn **F5** (hoặc nút Start màu xanh lá trên toolbar).
- IDE chuyển từ **Design Mode** sang **Run Mode**.
- Cửa sổ ứng dụng `ASimpleApp` xuất hiện ở tiền cảnh (foreground), chạy mượt mà độc lập.

Đặc trưng trạng thái Run Mode:

Trong lúc ứng dụng đang chạy, hầu hết các công cụ thiết kế trên Toolbox và Form Designer bị mờ (disabled) để bảo vệ tính toàn vẹn của mã nguồn.



Hình 2.35: Visual Studio IDE ở chế độ Run Mode (Giáo trình Deitel, trang 24).

Hands-on Lab 3: Xây dựng Hoàn thiện ‘ASimpleApp‘ (20 Phút)

NHIỆM VỤ THỰC HÀNH CÁ NHÂN:

1. Tự thực hiện trọn vẹn 10 bước đã học trên máy tính phòng máy.
2. Tạo Form có tiêu đề `"A Simple App"`, màu nền `LightCyan`.
3. Thêm Label có chữ `"Welcome to Visual C#!"`, Font Segoe UI 24pt Bold, căn lề `MiddleCenter`.
4. Thêm PictureBox hiển thị hình ảnh tự chọn từ máy tính, thiết lập `SizeMode = StretchImage`.
5. Nhấn **F5** chạy thử, kiểm tra căn lề và độ sắc nét.
6. Gọi giảng viên chấm điểm thực hành tại chỗ.

TIÊU CHÍ ĐÁNH GIÁ (RUBRICS):

- **Đúng bố cục (40%):** Label ở trên, PictureBox ở dưới cân đối.
- **Thuộc tính chuẩn (40%):** Đúng màu `LightCyan`, font 24pt bold, ảnh vừa khít không bị cắt xén méo mó.
- **Tác phong chuyên nghiệp (20%):** Lưu trữ dự án đúng cấu trúc thư mục quy định, thao tác phím tắt nhanh nhẹn.

Checkpoint 3 & Đánh giá Checklist Chuẩn Đầu Ra (CLOs)

Câu hỏi Checkpoint 3:

Khi muốn hình ảnh hiển thị trong PictureBox tự động co giãn để vừa khít hoàn toàn khung điều khiển, ta thiết lập thuộc tính `SizeMode` mang giá trị nào?

(A) `Normal`

(B) `AutoSize`

(C) `StretchImage`

(D) `CenterImage`

Checkpoint 3 & Đánh giá Checklist Chuẩn Đầu Ra (CLOs)

Câu hỏi Checkpoint 3:

Khi muốn hình ảnh hiển thị trong PictureBox tự động co giãn để vừa khít hoàn toàn khung điều khiển, ta thiết lập thuộc tính `SizeMode` mang giá trị nào?

(A) `Normal`

(B) `AutoSize`

(C) `StretchImage`

(D) `CenterImage`

Đáp án đúng: C (`StretchImage`). Giá trị này ép ảnh co giãn theo đúng kích thước khung hình.

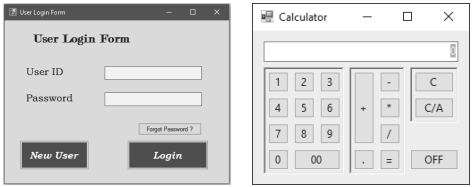
Checklist Chuẩn Đầu Ra Buổi học:

- ☒ **CLO 2.1:** Hiểu rõ bản chất IDE & RAD.
- ☒ **CLO 2.2:** Phân biệt cấu trúc `.sln`, `.csproj`, `Form.cs`, `Form.Designer.cs`.
- ☒ **CLO 2.3:** Sử dụng thành thạo Menus, Toolbars và phím tắt (F5, F4, Ctrl+Shift+S).
- ☒ **CLO 2.4:** Làm chủ Solution Explorer, Toolbox và Properties Window.
- ☒ **CLO 2.5:** Tự tay xây dựng thành công ứng dụng GUI hoàn chỉnh có văn bản và hình ảnh.

Bài tập Về Nhà: Thử thách Thiết kế 5 Giao diện Thực tế

Nhiệm vụ tự học (Giáo trình Deitel, trang 29–31):

1. **Bài 2.7 (Stop Watch GUI):** Thiết kế giao diện Đồng hồ bấm giờ có các nút Start, Stop, Reset (Hình 2.36).
2. **Bài 2.8 (Login Form GUI):** Thiết kế màn hình Đăng nhập có Username, Password, CheckBox Remember (Hình 2.37).
3. **Bài 2.9 (Calculator GUI):** Thiết kế bàn phím Máy tính bỏ túi với lưới các nút số 0–9 và phép tính (Hình 2.38).
4. **Bài 2.10 (Alarm Clock GUI):** Thiết kế giao diện Đồng hồ báo thức hẹn giờ (Hình 2.39).
5. **Bài 2.11 (Radio GUI):** Thiết kế máy nghe đài Radio có dò kênh và chỉnh âm lượng (Hình 2.40).



Hình 2.37 & 2.38: Giao diện Màn hình Đăng nhập và Máy tính bỏ túi.

Chuẩn bị cho Chương 3:

Đọc trước Chương 3: *Introduction to C# App Programming* – Bắt đầu viết mã lệnh xử lý số học, biến số và điều khiển luồng!